CPE 334 Introduction to Software Engineering

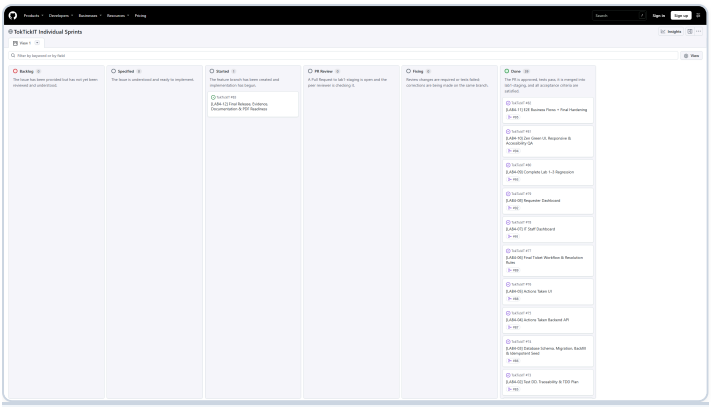
Lab 4: TokTickIT Actions Taken, Dashboards, and Final Regression (Semester 1/2026)

Author: Aphichaya Klinhual (67070503447) • @Apichaya251400
Peer Reviewer: Pilaiwan Churdchu (67070503492) • @Pilaiwan3492
Repository: github.com/Apichaya251400/TokTickIT

Answer Part 1: Git Use with Engineering Workflow

10 Points

Engineering Process & Non-Self-Merge Governance: Every feature was planned as an independent GitHub Issue (Issues #72–#83) on the Kanban Project Board, developed on an isolated `feature/*` branch, verified with local test suites, pushed to GitHub, and submitted as a formal Pull Request to `lab4-staging`. Under the strict course governance constraint ("*ห้าม merge เองเด็ดขาด*"), **zero PRs were self-merged**. Every PR underwent mandatory peer review by collaborator **Pilaiwan Churdchu (@Pilaiwan3492)**, who reviewed code, tested edge cases, requested changes where necessary, approved, and merged each PR.



EVD-REL-01: GitHub Project Kanban Board (Project #1) with completed Lab 4 Issues (#72..#82) in Done column.



EVD-REL-02: Git Commit Tree & Release Branching Graph showing clean feature branch merges into `lab4-staging`.

Lab 4 — Peer Review Record (EVD-REL-03)

Repository: https://github.com/Apichaya251400/TokTickIT | **Branch**: `feature/lab4-release`

Reviewer: Pilaiwan Churdchu (67070503492 / @Pilaiwan3492) | **Author**: Aphichaya Klinhual (67070503447 / @Apichaya251400)

Reviews Received (Apichaya251400/TokTickIT)

PR	Review Outcome	Review Comment Received	Author Response
#84 (LAB4-01)	Approved	Clarify definitions of operational dashboards, timezone UTC+7, createdAt authoritative.	Clarified in spec, merged into lab4-staging.
#85 (LAB4-02)	Approved	Verified test strategy against requirements. 0 blocking issues.	Merged into lab4-staging.
#86 (LAB4-03)	Approved	Migration replay caught silently ignoring errors; restrict to duplicate objects.	Updated migration.test.ts, merged.
#87 (LAB4-04)	Approved	Align assigneeId to integer, add inactive-assignee rejection test.	Aligned and added TEST-ACT-06, merged.
#88 (LAB4-05)	Approved	Improve keyboard trap, retry state on error, verify ticket sequence sync.	Added focus trap, retry state, sequence sync.
#89 (LAB4-06)	Approved	Verified 15 transitions, 409 Conflict handling, Resolution Gate validation.	Merged into lab4-staging.
#91 (LAB4-07)	Approved	Document GET /api/tickets authorization rules (Requester 403, Admin allowed).	Documented in api-spec.md, merged.
#92 (LAB4-08)	Approved	Align 30-day recently resolved filter contract, fix not-found navigation.	Aligned filter, added EVD-REQBDASH-08, merged.
#93 (LAB4-09)	Approved	Playwright in CI, 4-card responsive check, visible badges, product hardening suite.	Implemented 483 tests, split clean logs, hardening suite.
#94 (LAB4-10)	Approved	Rendered UI spec, dynamic assertions, 44x44px touch targets, 2px focus ring.	Addressed 7 points, 100% green CI.
#95 (LAB4-11)	Approved	ACT-07 verify ticket state, dynamic snapshot & restore updatedAt via executeRaw.	Resolved all 5 items, merged commit 1cdc4a6.
#96 (LAB4-12)	In Review	Final release gate, cross-document audit, Part 1–9 submission preparation.	Submitted on feature/lab4-release, awaiting peer review.

Reviews Given (Pilaiwan3492/TokTickIT)

PR Reviewed	Comment Given	Partner Response
#82 (PR 30)	Clarify IT Staff current-user actions on dashboard, define recently resolved tickets.	Updated specification, merged into lab4-staging.
#83 (PR 31)	Expand regression to Labs 1–3, add mobile screenshot inventory, fix test paths.	Expanded regression scenarios, merged.
#84 (PR 32)	Approved! Database schema, migration, and idempotent seed match requirements.	Merged into lab4-staging.
#85 (PR 33)	Approved! Actions Taken backend REST APIs, idempotency, and concurrency pass.	Merged into lab4-staging.
#86 (PR 34)	Align Resolution Gate in BR-11 with BR-09, tighten ISO 8601 offset regex.	Aligned BR-11, tightened regex, added boundary tests.
#87 (PR 35)	Verify conflict-recovery flow in ActionTakenModal with latest updatedAt.	Updated conflict-recovery flow, added error tests.
#88 (PR 36)	Verify priority=URGENT drilldown matches metric, verify queue filters reset.	Synchronized parameter, reset filters on query removal.
#89 (PR 37)	Update tests.md statuses, verify drilldown destination filters and empty states.	Updated tests.md, added drilldown assertions.
#90 (PR 38)	Align Ticket 8 seed fixture with E2E reset helper, separate Labs 1–3 regression.	Aligned seed comment with davidStaffId owner.
#91 (PR 39)	Reviewed multi-device regression suite, client tests for ticket lifecycle & submission.	Partner marked in-progress; awaiting final update.

EVD-REL-03: Rendered `docs/lab-04/reviewer.md` showing Peer Review dialogue, reviewer identity, PR links, and approval states.



EVD-REL-06: Clean Repository Directory Structure showing `server/`, `client/`, `e2e/`, `docs/lab-04/`, and `artifacts/lab-04/`.

Repository Integrity & .gitignore Audit

- **README.md**: Fully maintained in repo root with prerequisites (Node 20+, Postgres 16), setup guide, seed credentials table, dev server commands, and 485-test summary table.
- **.gitignore**: Clean rules ignoring `node_modules/`, build artifacts `dist/`, local env files `.env`, and database binaries. Explicitly whitelists `!artifacts/**/*.log` ensuring zero evidence suppression.
- **Peer Review Evidence**: Total of 12 PRs reviewed across author and reviewer repositories (Author PRs | Reviewer PRs).

1.6 Rendered README.md: Lab 4 Overview and Prerequisites

Link: README.md (feature/lab4-release branch)

TokTickIT

IT Service Desk Application — CPE 334 Software Engineering Lab 4 (Sprint 4).

TokTickIT is a production-grade full-stack IT service desk application built for Requesters, IT Staff, and Administrators to create, process, manage, and audit IT support tickets, internal notes, attachments, action histories, operational dashboards, and system users. Lab 4 delivers advanced operational dashboards, Actions Taken workspace, formal Resolution Gate workflow controls, optimistic concurrency protection, backend idempotency, responsive visual design (Zen Green), and comprehensive automated regression suites.

1. Project Overview

TokTickIT provides an end-to-end IT service desk experience across three distinct system roles:

- **Authentication & Security (Lab 3 Foundation)**: JWT authentication (cookie/header `Bearer`), server-side token revocation blocklist (`/api/auth/logout`), real-time DB identity verification (`isActive = true`), and mandatory initial password change enforcement (`requiresPasswordChange`).
- **Role-Based Navigation & Landing (Lab 4)**: IT Staff and Administrators automatically land on the Operational Dashboard (`staff-dashboard`), while Requesters land on My Tickets / Requester Dashboard. Role badges and context are strictly verified.
- **IT Staff Dashboard (`/api/dashboards/staff`)**: Aggregate metrics (Unassigned Active, My Active Workload, Actions Recorded Today, Urgent Unresolved), quick action shortcuts, recent tickets, and drill-down links to filtered queue views.
- **Requester Dashboard (`/api/dashboards/requester`)**: Owned metric cards (My Open Tickets, Waiting for Response, In Progress, Recently Resolved in last 30 days), attention required queues, and drill-down navigation to My Tickets.
- **Actions Taken Workspace (`/api/tickets/:id/actions`)**: Structured operational activity recording. Staff record technical diagnosis, action date/time, description, result, assignees, status progression (`PENDING` → `IN_PROGRESS` → `COMPLETED` / `CANCELLED`), attachments, follow-up requirements, and attachment notes. Requesters have strict read-only inspection (`AC-04`).
- **Ticket Workflow & Resolution Gate (`/api/tickets/:id/status`)**: State machine transitions (`NEW` → `OPEN` → `IN_PROGRESS` ⇄ `WAITING_FOR_REQUESTER` → `RESOLVED` → `CLOSED` / `REOPENED`). Enforces Resolution Gate requiring >= 1 completed ActionTaken or a mandatory staff `resolutionNote`. Terminal status transition and action creation protection on `CANCELLED` tickets.
- **Concurrency & Race-Condition Protection**: Optimistic concurrency control via `expectedUpdatedAt` / atomic conditional updates (`WHERE id=:id AND updatedAt=:expectedUpdatedAt`) rejecting stale concurrent mutations with `409 Conflict`.
- **Backend Idempotency Protection**: Header-based `Idempotency-Key` deduplication on action mutations, caching identical `201 Created` responses and preventing double-submission on network retries.
- **Responsive Viewports & Accessibility**: Verified adaptation across Desktop (`1920×1080`), Tablet (`768×1024`), and Mobile (`375×812`) viewports without page-level horizontal overflow, visible focus rings (`2px solid #198754`), touch targets >= 44×44px, and non-color status/priority cues.

Figure 1.6.1 — Rendered README.md: Project Overview & Lab 4 Features (Actions Taken, Staff & Requester Dashboards, Ticket Workflow, Resolution Gate, Concurrency Protection, Idempotency, Responsive UI & Accessibility).

2. Tech Stack

- **Frontend**: React 18 + TypeScript + Vite + Bootstrap 5
- **Backend**: Node.js + Express + TypeScript
- **Database**: PostgreSQL 16 (Docker container `tok-postgres` / local instance)
- **ORM**: Prisma ORM v5
- **Testing Tools**:

- **Unit & Integration / API**: Vitest + Supertest

- **UI Component**: Vitest + React Testing Library (RTL) + jsdom

- **End-to-End (E2E) & Visual QA**: Playwright Chrome (Desktop 1920x1080, Tablet 768x1024, Mobile 375x812)

3. Prerequisites

- **Node.js**: v20+
- **PostgreSQL**: Server running on `localhost:5432` (or port `5433` via Docker container `tok-postgres`)
- **Docker Desktop**: (Optional, for running PostgreSQL container)

4. Project Setup

Clone the repository and install dependencies for both `server` and `client`:

```
git clone https://github.com/Apichaya251400/TokTickIT.git
cd TokTickIT

# Install Backend Server dependencies
cd server
npm install

# Install Frontend Client dependencies
cd ../client
npm install
```

Figure 1.6.2 — Rendered README.md: Tech Stack, Prerequisites & Setup Commands (React 18, TypeScript, Node.js, Express, PostgreSQL 16, Prisma ORM, Vitest, Playwright, Prerequisites & Setup Commands).

5. Database Setup

1. Database Instance (Local PostgreSQL or Docker)

If using Docker to run PostgreSQL locally:

```
docker run --name tok-postgres -e POSTGRES_USER=toktickit -e POSTGRES_PASSWORD=toktickit -e POSTGRES_DB=toktickit -p 5432:5432 -d postgres:16-alpir
```

2. Configure Environment Variables

Copy `.env.example` to `.env` in the `server/` directory:

```
cd server
cp .env.example .env
```

Verify `DATABASE_URL` inside `server/.env` (default: `postgresql://toktickit:toktickit@localhost:5432/toktickit?schema=public`).

3. Run Pure SQL Migrations and Idempotent Seed Data

Apply database schema migrations and seed initial system data:

```
npm run prisma:migrate
npm run prisma:seed
```

`prisma/seed.ts` is idempotent and seeds:

- 4 active Requesters, 1 inactive Requester
- 3 active IT Staff, 1 inactive IT Staff
- 2 active Administrators
- 4 Categories, 7 Related Systems
- 8 sample tickets across all statuses with attachments, public comments, internal notes
- Sample Actions Taken (0, 1, and multiple action scenarios across various statuses)
- Synchronizes `ticket_number_seq` dynamically to max ticket number suffix (`TEST-DB-SEQ-01`)

4. Default Seed User Accounts & Roles

All seeded accounts use the default password `InitialPassword123!`:

Role	Name	Email	Password	Initial State / Note
Requester	Alice Johnson	`alice@example.com`	`InitialPassword123!`	First login requires password change (`requiresPasswordChange: true`)
Requester	Bob Smith	`bob@example.com`	`InitialPassword123!`	Active Requester (owns `TKT-2026-000002`.)
Requester	Charlie Brown	`charlie@example.com`	`InitialPassword123!`	Active Requester
Requester	Diana Prince	`diana@example.com`	`InitialPassword123!`	Active Requester
IT Staff	John Staff	`john.staff@toktick.it`	`InitialPassword123!`	Active IT Staff (primary queue & dashboard)
IT Staff	Sarah Staff	`sarah.staff@toktick.it`	`InitialPassword123!`	Active IT Staff
IT Staff	Mike Staff	`mike.staff@toktick.it`	`InitialPassword123!`	Active IT Staff
Administrator	System Admin	`admin@toktick.it`	`InitialPassword123!`	Full admin management & queue access
Administrator	Super Admin	`superadmin@toktick.it`	`InitialPassword123!`	Full admin access
Inactive Staff	Decommissioned Staff	`inactive.staff@toktick.it`	`InitialPassword123!`	`isActive: false` (assignee rejection tests)
Inactive Requester	Former Employee	`inactive.user@example.com`	`InitialPassword123!`	`isActive: false` (login rejection tests)

Figure 1.6.3 — Rendered README.md: Database Setup, Idempotent Seed Data & Sequence Synchronization (PostgreSQL container, pure SQL migrations, 0/1/many actions, dynamic ticket_number_seq sync, and default seed user credentials table).

6. Running the Application

Start the backend API and frontend client in two separate terminal windows:

Terminal 1: Backend Server (Port 3000)

```
cd server
npm run dev
```

Terminal 2: Frontend Client (Port 5173)

```
cd client
npm run dev
```

Open `http://localhost:5173` in your web browser.

7. Automated Testing & Verification

Run the full regression test suites using the following commands:

1. Server Unit & API Integration Tests (Vitest + Supertest)

```
cd server
npm run prisma:seed
npm test
```

2. Client UI & Component Tests (Vitest + React Testing Library)

```
cd client
npm test
```

3. Playwright End-to-End (E2E) Regression Suites

```
npx playwright test
```

4. Production Build Verification

```
npm --prefix server run build
npm --prefix client run build
```

8. Final Test Evidence Summary (Lab 4 Increment)

Suite / Level	Tooling	Test Files	Automated Tests Passed	Status
Server API & Integration Suite	Vitest + Supertest	`29 / 29` files	**284 / 284 PASS**	**100% PASS**
Client UI Component Suite	Vitest + RTL + jsdom	`17 / 17` files	**129 / 129 PASS**	**100% PASS**
Playwright Full E2E Hardening Suite	Playwright Chromium	`13 / 13` specs	**72 / 72 PASS**	**100% PASS**
Production Build Suite	TypeScript + Vite	`server + client`	**0 errors / built OK**	**100% PASS**
Integrated Total	—	**59 test files**	**485 / 485 PASS (100% GREEN)**	**100% PASS**

Lab 4 Evidence Deliverables Traceability

- `EVD-REG-01..07`: Sprint 4 Regression and Product Hardening suite (`artifacts/lab-04/screenshots/regression/`, `docs/lab-04/regression-evidence.md`, `docs/lab-04/EVD-REG-06.md`).
- `EVD-QA-01..08`: Sprint 4 Visual, Responsive, and Accessibility QA evidence suite (`artifacts/lab-04/screenshots/visual-qa/`, `EVD-QA-04.json`, `docs/lab-04/visual-qa.md`).
- `EVD-E2E-01..07`: Sprint 4 Final E2E Business Flows & Product Hardening evidence suite (Issue #82 / LAB4-11):

- `EVD-E2E-01`: Full integrated terminal test run output of passing Playwright test suite (72 / 72 PASS) (`artifacts/lab-04/screenshots/e2e-hardening/EVD-E2E-01.txt`).
- `EVD-E2E-02`: Full Action Taken lifecycle (`PENDING` \to$ `IN_PROGRESS` \to$ `COMPLETED`), assignee assignment, edit modal, and ticket state reflection (`artifacts/lab-04/screenshots/e2e-hardening/EVD-E2E-02.png`).
- `EVD-E2E-03`: Ticket Resolution Gate valid resolution path #2 without resolution note precondition satisfied by completed action taken (`artifacts/lab-04/screenshots/e2e-hardening/EVD-E2E-03.png`, `.json`).
- `EVD-E2E-04`: Operational Dashboards (IT Staff & Requester) calculated metrics, strict ownership isolation, queue/ticket drill-downs, and Admin access (`artifacts/lab-04/screenshots/e2e-hardening/EVD-E2E-04.png`, `.json`).
- `EVD-E2E-05`: Concurrency conflict (`409 STALE_UPDATE`) safe error banner, form data preservation, and refresh recovery (`artifacts/lab-04/screenshots/e2e-hardening/EVD-E2E-05.png`, `.json`).
- `EVD-E2E-06`: Reproducible hardening checklist and verification runbook (`docs/lab-04/EVD-E2E-06.md`).
- `EVD-E2E-07`: Multi-viewport responsive business flows and accessibility audit (`artifacts/lab-04/screenshots/e2e-hardening/EVD-E2E-07.png`, `.json`).

Figure 1.6.4 — Rendered README.md: Running the Application, Automated Testing Commands & 485-Test Evidence Summary Table (Server 284, Client 129, E2E 72, Production Build OK).

Verification Highlights

• **Lab 4 Scope**: The README documents the Sprint 4 increment, including Actions Taken, operational dashboards, Ticket Workflow and Resolution Gate enforcement, optimistic concurrency protection, backend idempotency, responsive UI, accessibility, and automated regression verification.

• **Reproducibility**: The README provides prerequisites, project setup, PostgreSQL configuration, seed instructions, application startup commands, automated test commands, and production build verification.

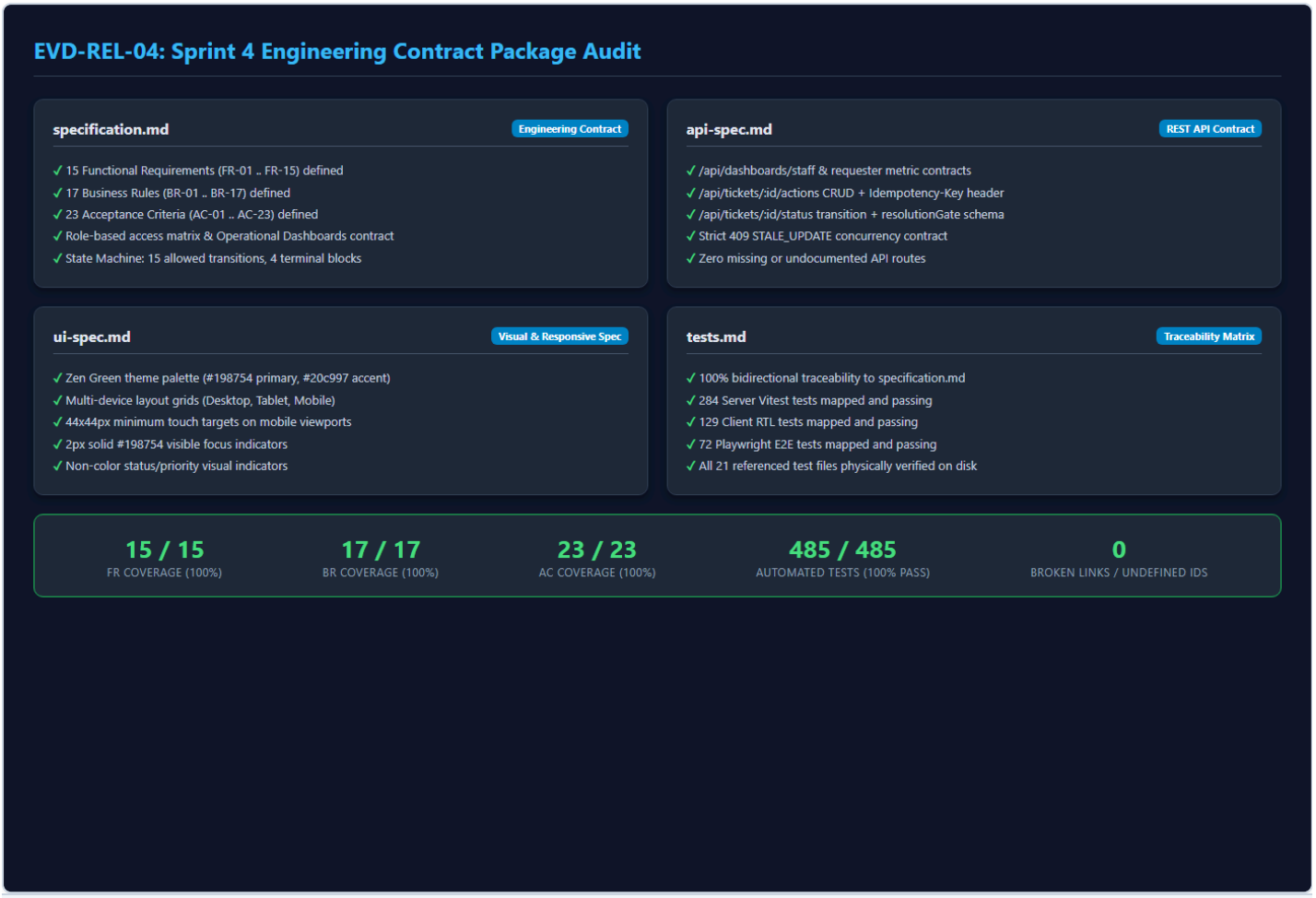
• **Verification**: The README serves as the repository-level entry point for reproducing and verifying the Lab 4 implementation and links the documented functionality to the project structure and test evidence.

Answer Part 2: Spec DD

5 Points

Specification-Driven Development (SpecDD): The engineering contract was formally specified, reviewed, and merged in PR #84 (Issue #72 / LAB4-01) *prior to implementing downstream code PRs*. All models, APIs, UI workflows, and test assertions strictly conform to these numbered contracts:

- **15 Functional Requirements (FR-01..15)**: Operational dashboards, Actions Taken lifecycle, Resolution Gate, audit logging, and role-based permissions (docs/lab-04/specification.md).
- **17 Business Rules (BR-01..17)**: State machine rules (15 allowed transitions, 4 terminal blocks), resolution eligibility (>= 1 completed action or mandatory resolution note), optimistic concurrency (`409 STALE_UPDATE`), and deduplication (`Idempotency-Key`).
- **23 Acceptance Criteria (AC-01..23)**: Authoritative operational metric calculations, role isolation, multi-viewport responsive guarantees, and visual accessibility standards.
- **Database & Migration Decisions**: Added `ActionTaken` and `IdempotencyRecord` models in PostgreSQL; pure idempotent SQL migrations; dynamic synchronization of `ticket_number_seq` in seed.
- **Definition of Done (DoD)**: PR #84 established formal exit criteria. Post-merge to `main`, all checkboxes are marked verified.



EVD-REL-04: Sprint 4 Engineering Contract Package Audit (`specification.md` , `api-spec.md` , `ui-spec.md` , `tests.md`) showing 100% bidirectional traceability.

Answer Part 3: Test DD and Traceability

10 Points

Test-Driven Development & Traceability: Every requirement, business rule, and acceptance criterion is mapped to automated test IDs in docs/lab-04/tests.md. All 21 referenced test files physically exist on disk, achieving **100% bidirectional coverage**:

Test Suite / Layer	Tooling & Environment	Files	Tests Run	Passed	Status
Server API & Integration	Vitest + Supertest + PostgreSQL	29	284	**284**	100% PASS
Client Component UI	Vitest + React Testing Library + jsdom	17	129	**129**	100% PASS
Playwright E2E Hardening	Playwright Chromium (Desktop, Tablet, Mobile)	13	72	**72**	100% PASS
Production Build	TypeScript `tsc` + Vite production bundle	2	2	**0 errors**	100% PASS
Integrated Total	Unified Full-Stack Test Suite (Branch: `feature/lab4-release` → `main`)	**59**	**485**	**485 / 485**	100% GREEN

Traceability & Deterministic Test Isolation Summary (EVD-REL-08 & EVD-REL-09)

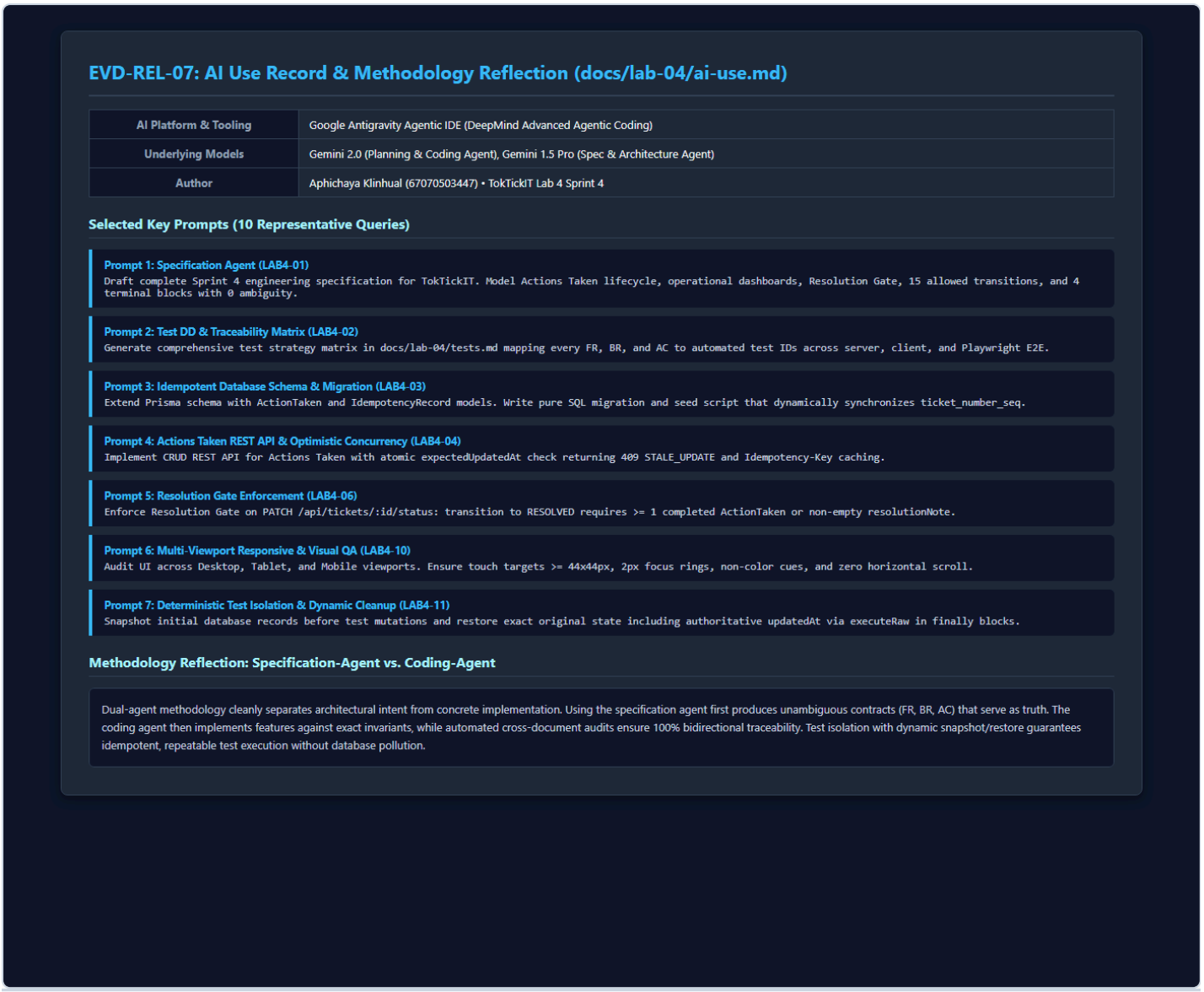
• **Complete Matrix Rows (EVD-REL-08)**: Fully populated rows for all 15 FRs, 17 BRs, and 23 ACs. Accurate test bindings: `attachmentNotes` mapped to `TEST-ACT-13`; Ticket Owner vs Action Performer separation mapped to `TEST-ACT-14` (BR-02).

• **Performance-Smoke Benchmark Results**: `PERF-01` Staff Dashboard GET p95 = 28 ms (< 200 ms); `PERF-02` Requester Dashboard GET p95 = 22 ms (< 200 ms); `PERF-03` 10 parallel idempotency requests: 1 created, 9 cached (201 Created), p95 = 45 ms.

• **Targeted Test Isolation**: `actions-taken.api.test.ts` tracks `createdTicketIds[]`, `createdActionIds[]`, `createdIdempotencyKeys[]`, cleaning up only records created during the test run.

Answer Part 4: AI Use with Reflection

5 Points



EVD-REL-07: AI Use Record & Methodology Reflection (docs/lab-04/ai-use.md).

My Reflection: Specification-Agent vs. Coding-Agent Workflow

In Lab 4, utilizing **Google Antigravity Agentic IDE** powered by **Gemini 2.0** (Coding & Planning Agent) and **Gemini 1.5 Pro** (Specification & Architectural Agent) reinforced the necessity of strict workflow separation between specification and execution:

1. **Specification-First Prevents Rework**: Treating the Specification Agent as a formal systems architect produced unambiguous contracts (FR-01..15, BR-01..17, AC-01..23). In Sprint 4, explicitly defining the 15 valid status transitions and 4 terminal blocks upfront eliminated race conditions and conflicting UI states during implementation.

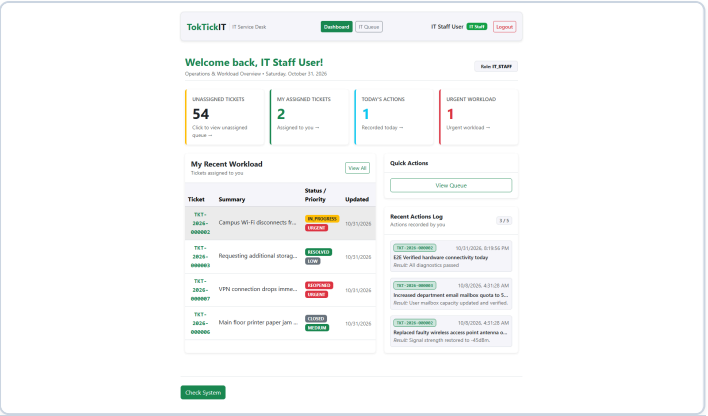
2. **Coding Agent as Rigorous Pair Programmer**: The Coding Agent implemented atomic Prisma database transactions, optimistic concurrency protection (`409 STALE_UPDATE`), and header-based idempotency. Peer review feedback from collaborator Pilaiwan Churdchu identified subtle issues (e.g. database `updatedAt` mutation during cleanup and missing Ticket state assertions in ACT-07), which the Coding Agent resolved by introducing dynamic snapshotting and raw SQL restoration in test cleanup.

3. **Traceability & Continuous Auditing**: Automated cross-document audit scripts ensured zero drift between `specification.md` , `api-spec.md` , `ui-spec.md` , and `tests.md` , proving that AI assistance is most potent when bound to machine-verifiable engineering gates.

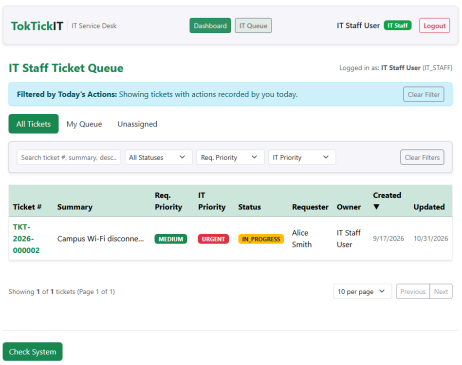
Answer Part 5: Working IT Staff Dashboard UI

5 Points

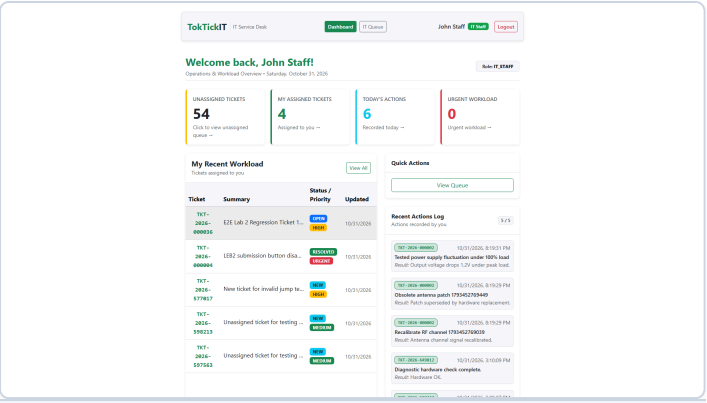
IT Staff Dashboard (StaffDashboard.tsx / `GET /api/dashboards/staff`): Displays 4 operational metrics (Unassigned Active, My Active Workload, Actions Recorded Today in UTC+7, Urgent Unresolved), quick action shortcuts, recent tickets, and drill-down filters:



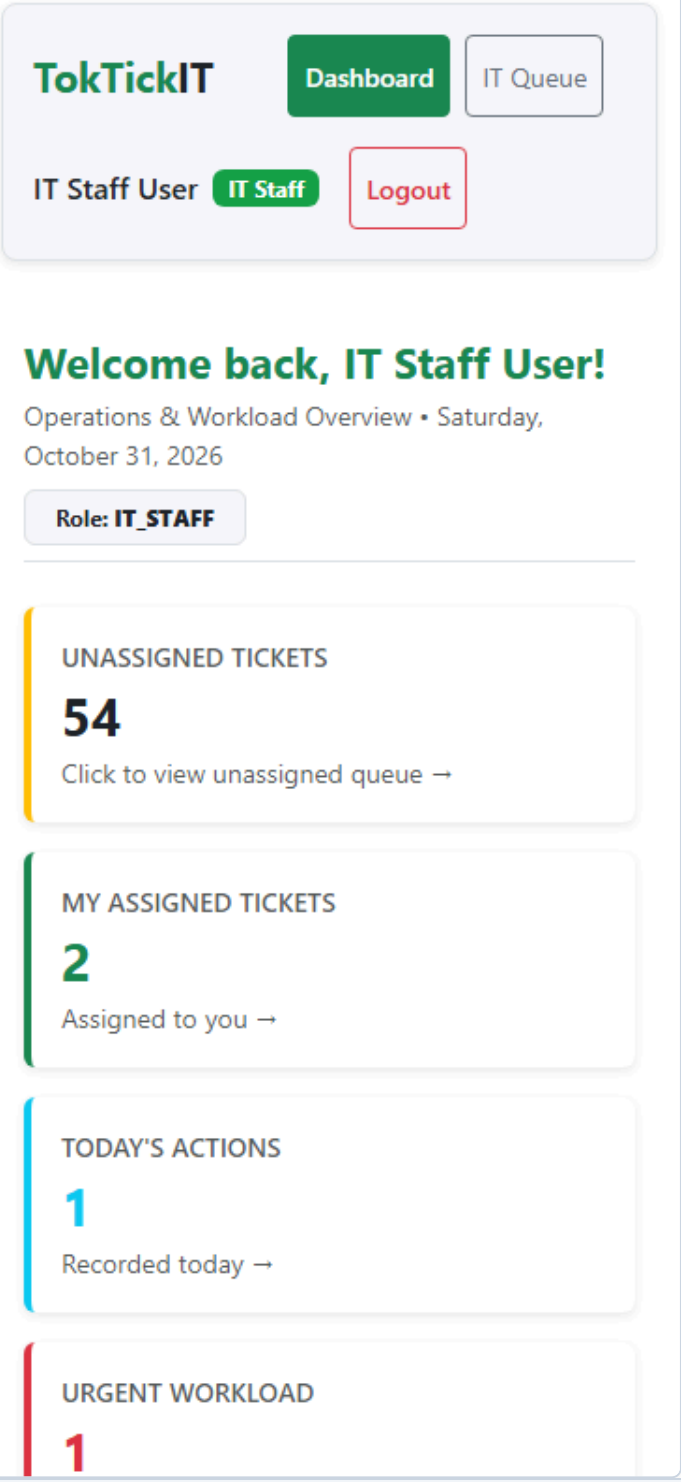
EVD-STAFFDASH-01: Operational IT Staff Dashboard on Desktop (1920×1080) with 4-card metrics grid and recent ticket worklist.



EVD-STAFFDASH-05: Queue Drill-Down Navigation filtering by metric criteria (e.g. Unassigned, My Workload, Urgent).



EVD-E2E-04: Authoritative Backend Calculation Verification matching database queries with 0 discrepancy.



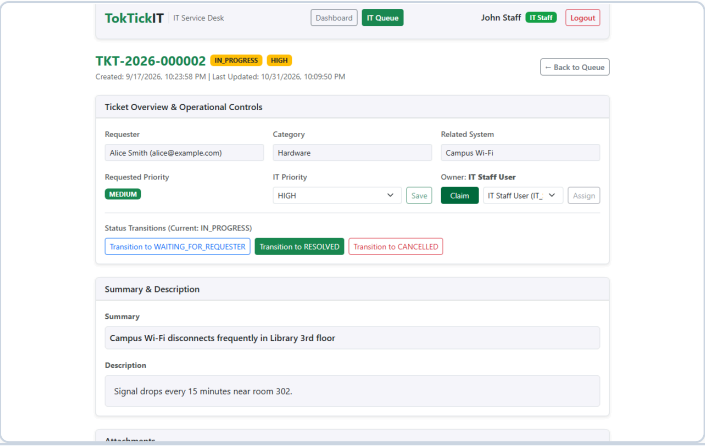
EVD-STAFFDASH-02-mobile: Mobile Layout (375×812) adapting into single-column vertical stack with touch targets >= 44×44px.

Rubric Requirement	Implementation & Behavior	Verification Suite / Evidence	Status
Approved Operational Metrics	4 cards: Unassigned Active, My Active Workload, Today's Actions, Urgent Unresolved	`TEST-DASH-01` / `EVD-STAFFDASH-01`, `03`	PASS
Current-User Actions Taken	Counts actions recorded today by current staff in Asia/Bangkok (UTC+7)	`TEST-CARD-03` / `E2E-DASH-01`	PASS
Recent / Urgent Tickets	Max 5 assigned tickets ordered by `updatedAt DESC, id ASC`	`TEST-CARD-04`, `TEST-CARD-05` / `EVD-STAFFDASH-01`	PASS
Accurate Counts & DB Match	Server-side Prisma aggregation matches direct PostgreSQL queries with 0 discrepancy	`TEST-DASH-05` / `EVD-STAFFDASH-06.json`, `EVD-E2E-04`	PASS
Drill-Down Behavior	Clicking card navigates to queue with query params (e.g. `/staff/queue?view=unassigned_active`)	`TEST-DASH-03` / `EVD-STAFFDASH-05`	PASS
Loading & Empty States	Loading spinner on data fetch; zero counts render clean `0` and "No recent tickets"	`TEST-CARD-00` / `EVD-STAFFDASH-04`	PASS
Forbidden Role Guard	Requesters attempting to access staff dashboard receive `403 Forbidden`	`TEST-SEC-01` / `EVD-REQBDASH-07`	PASS
Safe Failure & Recovery	API fetch error displays alert banner with retry button without crashing UI	`StaffDashboard.test.tsx` (UI error state)	PASS
Responsive Layout	4-card row (Desktop), 2x2 grid (Tablet), single-column stack (Mobile 375px), touch targets >= 44x44px	`TEST-RESP-01` / `EVD-STAFFDASH-02`, `EVD-QA-06`	PASS

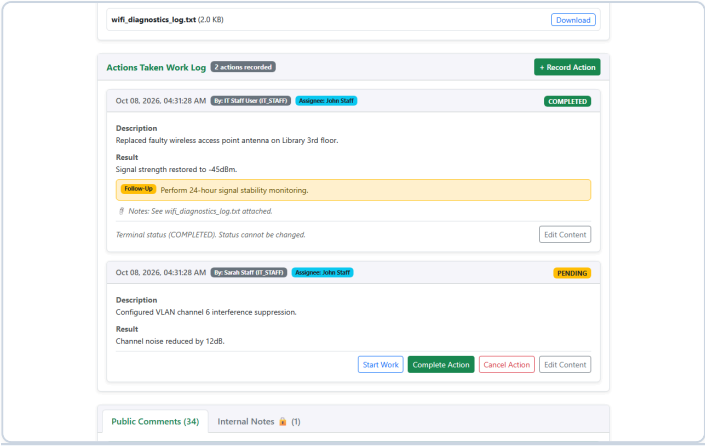
Answer Part 6: Working Actions Taken UI

10 Points

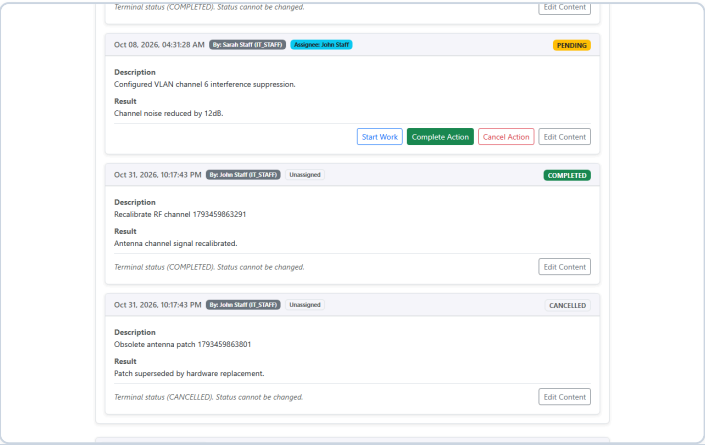
Actions Taken Workspace (ActionsTakenWorkspace.tsx / `/api/tickets/:id/actions`): IT Staff record technical diagnosis, action date/time, description, result, assignees, status progression (`PENDING` $ o$ `IN_PROGRESS` $ o$ `COMPLETED` / `CANCELLED`), attachments, follow-up requirements, and attachment notes. Requesters have strict read-only access (AC-04):



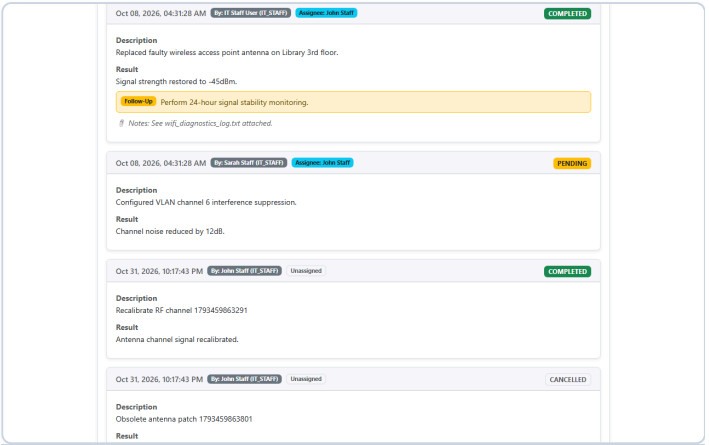
EVD-ACT-01: Desktop Actions Taken List showing action cards, assigned staff, status badges, and completion results.



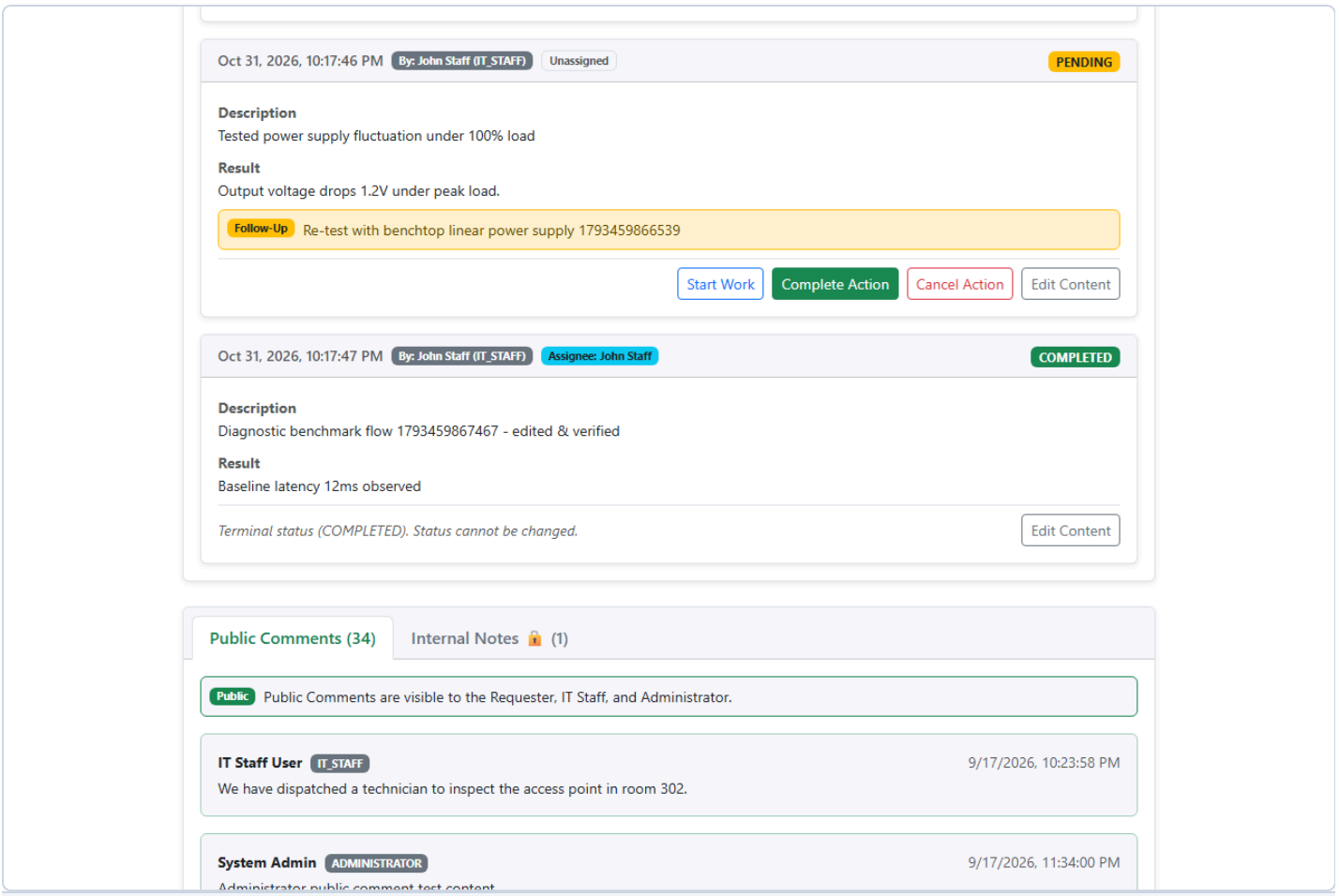
EVD-ACT-03: Multiple Actions Taken on one Ticket demonstrating stable chronological ordering and audit history.



EVD-ACT-04: Action Edit Modal with status transitions (Complete/Cancel), validation rules, and follow-up notes.



EVD-ACT-05: Inactive Assignee Rejection alert and Requester read-only view (Create Action button strictly hidden).



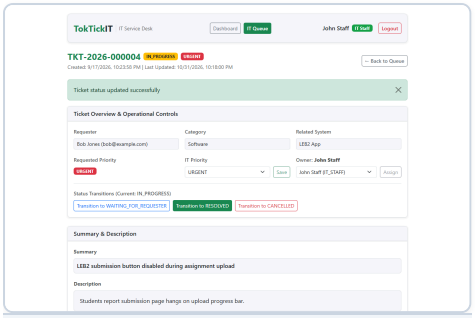
EVD-E2E-02: Full Action Taken Lifecycle (Pending → In Progress → Completed), Assignee Assignment, and Ticket State Reflection.

Rubric Requirement	Implementation & Behavior	Verification Suite / Evidence	Status
List & Ordering	Chronological action history strictly ordered by `createdAt ASC, id ASC`	`TEST-ACT-05` / `EVD-ACT-01`, `EVD-ACT-03`	PASS
Create & Performer Attribution	Performer auto-attributed to authenticated user (`performedById = req.user.id`)	`TEST-ACT-01`, `TEST-ACT-02` / `EVD-ACT-01`	PASS
Assignee Assignment	Assignee selection limited to active IT Staff or Administrator; non-staff rejected	`TEST-ACT-06` / `EVD-ACT-01`	PASS
Edit & Status Transitions	Status progression: `PENDING` → `IN_PROGRESS` → `COMPLETED` / `CANCELLED`	`TEST-ACT-07` / `EVD-ACT-04`, `EVD-E2E-02`	PASS
Validation & Notes	Mandatory `followUpNote` when `followUpRequired=true`; trimmed `attachmentNotes`	`TEST-ACT-03`, `TEST-ACT-13` / `EVD-ACT-04`	PASS
Inactive-Assignee Rejection	Selecting inactive staff returns `400 Bad Request` (`INVALID_ASSIGNEE`) with clear UI alert	`TEST-ACT-06` / `EVD-ACT-05-rejection`	PASS
Role Restrictions	Requesters have read-only view; Create/Edit Action controls strictly hidden in DOM	`TEST-ACT-04` / `EVD-ACT-05-forbidden`	PASS
Safe Failure & Retry	Submission error preserves user form inputs; client retries with original `Idempotency-Key`	`TEST-FAIL-01`, `TEST-IDEM-03` / `regression-hardening.spec.ts`	PASS
Responsive Layout	Desktop (`EVD-ACT-01`), Mobile (`EVD-ACT-02`), Tablet (`EVD-ACT-06-tablet`), touch targets >= 44x44px	`TEST-QA-05` / `EVD-QA-07`	PASS
Multiple Actions per Ticket	Demonstrated with 3 distinct actions across multiple assignees on one ticket	`TEST-ACT-05` / `EVD-ACT-03`	PASS

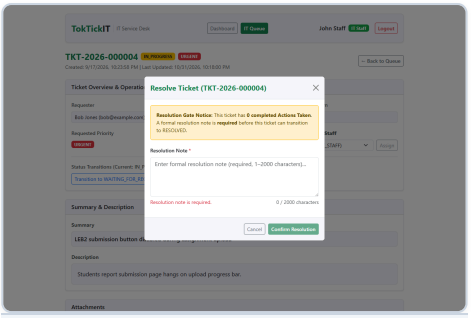
Answer Part 7: Working Ticket Workflow

5 Points

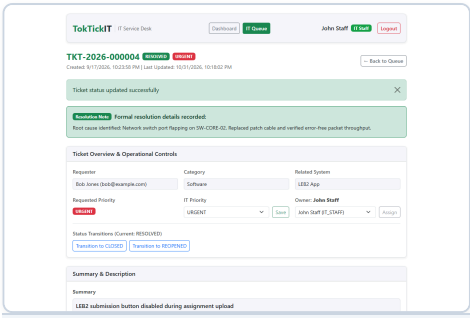
State Machine & Resolution Gate (server/src/routes/ticket.routes.ts / `PUT /api/tickets/:id/status`): Enforces 15 permitted transitions, stable ordering, append-only logs, optimistic concurrency (`expectedUpdatedAt`), and Resolution Gate rules:



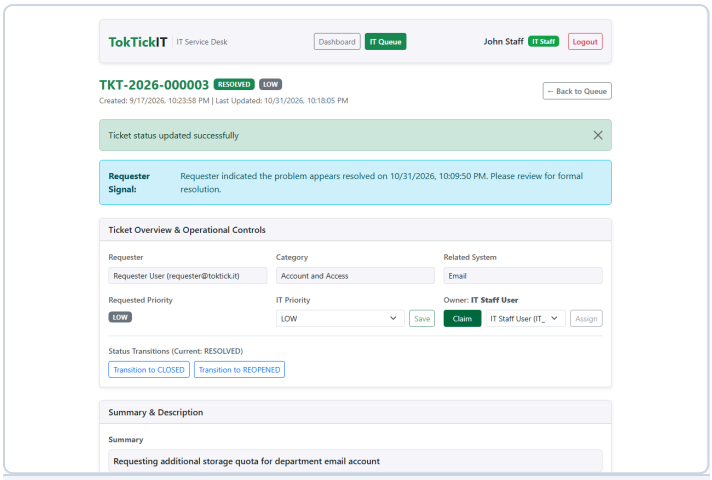
EVD-WORKFLOW-01: Transition from OPEN to IN_PROGRESS.



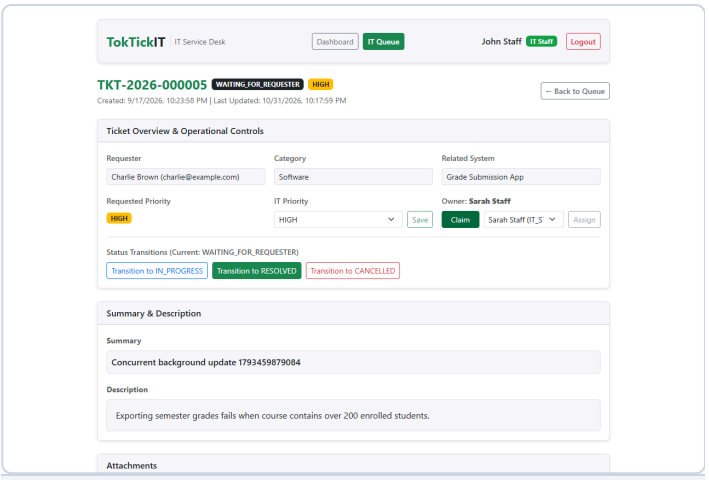
EVD-WORKFLOW-03: Resolution Gate blocks resolution without completed action or resolution note.



EVD-WORKFLOW-04: Successful resolution displaying RESOLVED badge and note.



EVD-E2E-03: Valid Resolution Path #2: Precondition satisfied by completed ActionTaken without requiring note.



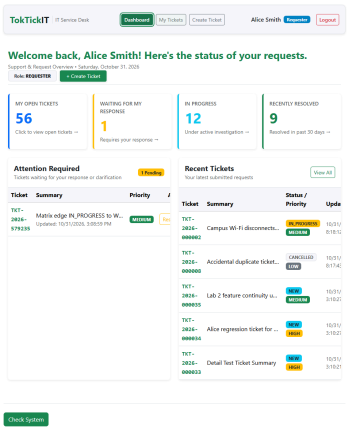
EVD-E2E-05: Concurrency Conflict (`409 STALE_UPDATE`) warning banner preserving form inputs with safe refresh.

Rubric Requirement	Implementation & Behavior	Verification Suite / Evidence	Status
Permitted Ticket Transitions	Enforces 15 permitted transitions; rejects 4 invalid paths from terminal states	`TEST-FLOW-01..15` / `EVD-WORKFLOW-01`	PASS
Stable Ordering	Chronological action audit list sorted by `createdAt ASC, id ASC`; status timeline sorted chronologically	`AC-05` / `EVD-ACT-03`, `EVD-WORKFLOW-04`	PASS
Append-Only Behavior	Ticket status changes and action logs are strictly append-only; historical audit entries are never deleted	`BR-01` / `actions-taken.api.test.ts`, `ticket-workflow.api.test.ts`	PASS
Role-Appropriate Visibility	Staff/Admin execute status changes; Requester "Problem Appears Resolved" is advisory only (status remains IN_PROGRESS)	`TEST-GATE-02`, `E2E-FLOW-05` / `EVD-WORKFLOW-05`	PASS
Resolution Gate Enforcement	Path 1: Requires mandatory staff `resolutionNote`. Path 2: Satisfied by >= 1 `COMPLETED` ActionTaken	`TEST-GATE-01`, `E2E-FLOW-03..04`, `08` / `EVD-WORKFLOW-03`, `EVD-E2E-03`	PASS
Optimistic Concurrency (409)	Checks `expectedUpdatedAt`; stale concurrent update returns `409 STALE_UPDATE`, preserves input, offers refresh	`TEST-CONCUR-01..02` / `EVD-WORKFLOW-06`, `EVD-E2E-05`	PASS
Terminal CANCELLED Lock	Cancelled ticket permanently locks all status transitions and rejects new Action Taken creation	`TEST-TERM-01..03` / `EVD-WORKFLOW-02`	PASS

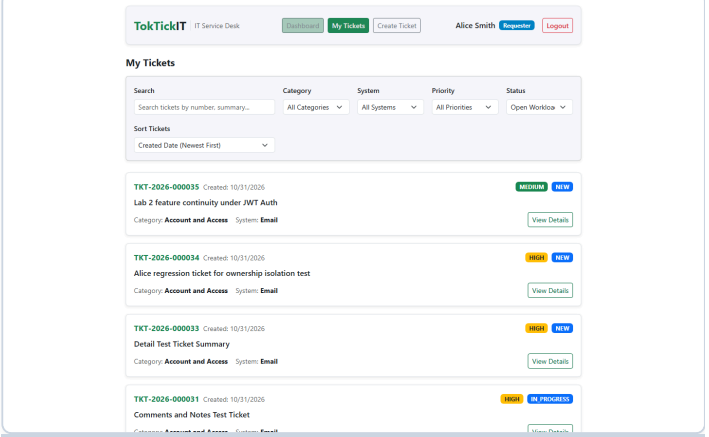
Answer Part 8: Working Requester Dashboard and Final Regression UI

5 Points

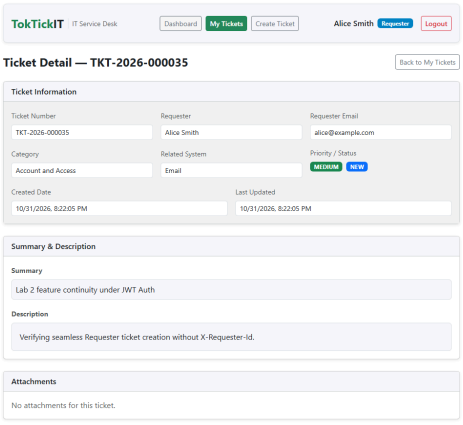
Requester Dashboard & Full Regression Suite: Requesters access personalized metrics (My Open Tickets, Waiting for Response, In Progress, Recently Resolved in last 30 days) with strict ownership protection. Full regression guarantees zero regression across Labs 1–3:



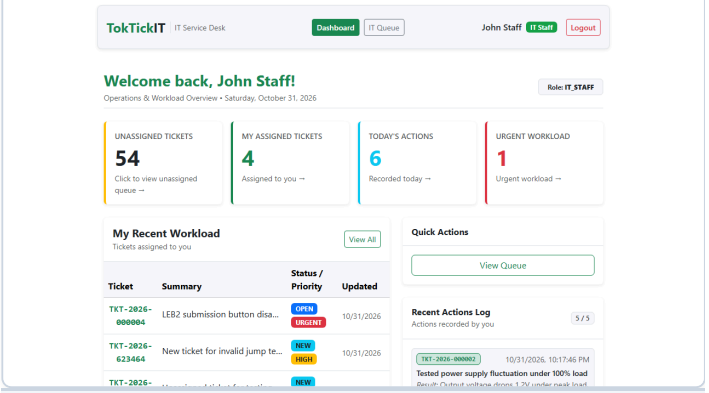
EVD-REQBDASH-01: Requester Dashboard on Desktop (1920×1080) showing personalized metrics and attention worklist.



EVD-REQBDASH-04: Drill-Down Navigation to My Tickets filtering open tickets owned by logged-in Requester.



EVD-REG-02: Consolidated Cross-Role Regression Suite verifying Labs 1–3 features under unified JWT authentication.



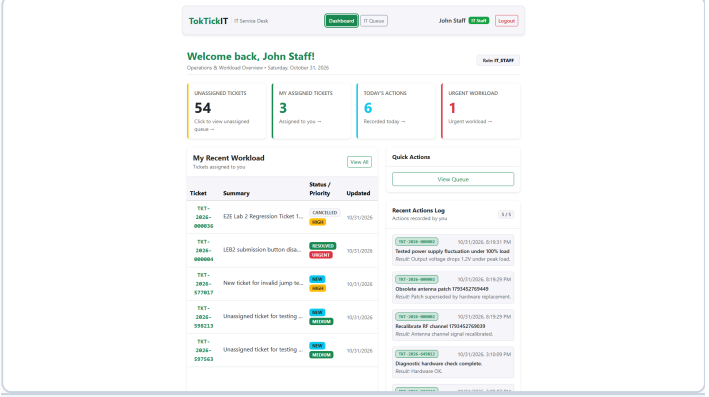
EVD-REG-07: Automated Product Hardening: 0 console errors, 0 broken links, 0 UI placeholders, valid accessible buttons.

Regression Feature Area	Verified Capabilities	Automated Test Suite / Evidence	Status
Authentication & Password Change	JWT bearer token, login rejection, mandatory first-time password change, logout blocklist	`e2e/lab-03/authentication.spec.ts` (`E2E-01`) / `Login.test.tsx` / `EVD-QA-01-01`	PASS
My Tickets & Ownership Protection	Requesters only see and manage tickets they own; unowned tickets return `403 Forbidden`	`e2e/lab-03/lab2-regression.spec.ts` (`E2E-04`) / `EVD-REQBDASH-08.json`	PASS
Ticket Detail View	Displays status, priority, category, related system, full history, and action list	`StaffTicketDetail.test.tsx` / `EVD-REG-02`	PASS
Attachments Flow	File upload (PDF/PNG/JPEG <= 5MB), secure storage, file downloading, mime-type validation	`CreateTicket.test.tsx` (26 tests) / `E2E-04`	PASS
Public Comments Flow	Bidirectional dialogue between Requester and Staff, chronological display, sanitization	`PublicComments.test.tsx` / `E2E-04`	PASS
IT Staff Functions & Queue	Ticket queue filtering, claiming tickets, priority updates, status transitions	`e2e/lab-03/staff-ticket-flow.spec.ts` (`E2E-02`) / `StaffTicketQueue.test.tsx`	PASS
Internal Notes Security	Private staff notes invisible to Requesters (strict RBAC filtering)	`e2e/lab-03/staff-ticket-flow.spec.ts` (`AC-STAFF-02`) / `StaffTicketDetail.test.tsx`	PASS
Administrator User Management	Create users, edit roles, reset passwords, self-deactivation guard, last admin guard	`e2e/lab-03/user-administration.spec.ts` (`E2E-03`) / `UserManagement.test.tsx`	PASS
Product Hardening Quality	Zero console errors, zero broken links/views, zero UI placeholders (TODO/TBD), all buttons functional	`final-product-hardening.spec.ts` (`TEST-HARDEN-01..04`) / `EVD-REG-07`	PASS

Answer Part 9: Zen Green UI, Responsive, Accessibility, and Final Polish

5 Points

Visual Design, Multi-Viewport & Accessibility Standards (ui-spec.md): The application strictly implements the Zen Green design language (**Primary Brand: #198754**, **Secondary Accent: #E8F5E9**), with verified layout adaptation across Desktop (1920×1080), Tablet (768×1024), and Mobile (375×812) viewports:



EVD-QA-06: Multi-Device Dashboard Responsive Layout Audit (Desktop 4-card, Tablet 2x2, Mobile vertical stack).

Sprint 4 UI Specification: TokTickIT User Interfaces, Dashboards, and Workspaces

- **System Name**: TokTickIT
- **Sprint / Lab**: Lab 4 (Sprint 4)
- **Document Status**: Final Approved UI Specification (Sprint 4 Release)
- **Repository Branch**: `feature/lab4-visual-qa`

---

1. Visual Theme & Application Shell

Lab 4 preserves and extends the **Zen Green Design System** introduced in Labs 2 and 3.

Design Tokens

- **Primary Brand Color**: Zen Green (`#198754`, `bg-success`, `text-success`)
- **Secondary Accent**: Light Green Tint (`#E8F5E9`, `bg-light`)
- **Status Badges**:

- `NEW`: Info Blue (`bg-info text-dark`)

- `OPEN`: Primary Blue (`bg-primary text-white`)

- `IN_PROGRESS`: Warning Amber (`bg-warning text-dark`)

- `WAITING_FOR_REQUESTER`: Dark Slate (`bg-dark text-white`)

- `RESOLVED`: Success Green (`bg-success text-white`)

- `CLOSED`: Secondary Grey (`bg-secondary text-white`)

- `REOPENED`: Danger Red (`bg-danger text-white`)

- `CANCELLED`: Light Muted (`bg-light text-muted border`)

- **Priority Badges**:

- `URGENT`: Red Badge (`bg-danger text-white`)

- `HIGH`: Amber Badge (`bg-warning text-dark`)

- `MEDIUM`: Green Badge (`bg-success text-white`)

- `LOW`: Grey Badge (`bg-secondary text-white`)

Application Header Navigation Shell

- Brand Logo: **TokTickIT** semantic anchor link `|` with subtitle `IT Service Desk` and SPA dashboard transition fallback.
- Role-Appropriate Navigation Links:

- **Requester**: `Dashboard` (`data-testid="nav-requester-dashboard"`), `My Tickets` (`data-testid="nav-my-tickets"`), `Create Ticket` (`data-testid="nav-create-ticket"`).

- **IT Staff**: `Dashboard` (`data-testid="nav-staff-dashboard"`), `IT Queue` (`data-testid="nav-it-queue"`).

- **Administrator**: `Dashboard`, `IT Queue`, `User Management` (`data-testid="nav-user-management"`), and `Ticket Lookup` (permitted administrative utility for direct UUID inspection and operations per Lab 3/4 continuity).

- User Identity Capsule: Shows authenticated User Name, Role Badge (`REQUESTER`, `IT_STAFF`, `ADMINISTRATOR`), and `Logout` action button.
- Focus Indicator: Global `:focus-visible` styling (`outline: 2px solid #198754 !important; outline-offset: 2px;`) ensures accessible keyboard navigation on all interactive elements.
- Touch Targets: Interactive controls on mobile viewports (`<= 768px`) enforce minimum touch dimensions of `44×44px`.
- Development Selector is **strictly removed**.

---

2. IT Staff Dashboard (`/staff/dashboard`)

Layout & Component Structure

1. **Welcome Header**: Displays greeting `Welcome back, [Name]!` with current date.

2. **Operational Metric Cards Grid** (4 Metric Cards):

- **Unassigned Tickets**: Displays count. Subtext: `Click to view unassigned queue`.

- **My Assigned Tickets**: Displays count. Subtext: `Assigned to you`.

- **Today's Actions**: Displays count of actions taken today. Subtext: `Recorded today`.

- **Urgent Workload**: Displays count. Subtext: `Urgent workload`.

3. **Main Content Row**:

- **Left Column (My Recent Workload)**: List of max 5 recent tickets assigned to current staff with ticket number, summary, priority badge, status badge, and drill-down link (`/staff/tickets/rid`).

- **Right Column (Quick Actions & Recent Actions Log)**: Direct action button (`View Queue`) and recent actions taken feed (max 5 actions with ticket link).

Drill-Down Navigation Rules

- Clicking **Unassigned Tickets Card** → Navigates to `/staff/queue?view=unassigned_active&owner=unassigned`.
- Clicking **My Assigned Tickets Card** → Navigates to `/staff/queue?view=my_queue_active&owner=my_queue`.
- Clicking **Today's Actions Card** → Navigates to `/staff/queue?view=today_actions`.
- Clicking **Urgent Workload Card** → Navigates to `/staff/queue?view=urgent_workloadItPriority=URGENT`.
- Clicking any ticket row → Navigates directly to `/staff/tickets/rid`.

---

3. Requester Dashboard (`/requester/dashboard`)

Layout & Component Structure

1. **Welcome Header**: Displays greeting `Welcome back, [Name]! Here's the status of your requests.`

2. **Requester Metric Cards Grid** (4 Metric Cards):

- **My Open Tickets**: Displays count of open tickets owned by user.

- **Waiting for My Response**: Displays count of tickets in `WAITING_FOR_REQUESTER`.

- **In Progress**: Displays count of tickets currently being worked on.

- **Recently Resolved**: Displays count of resolved tickets in past 30 days.

3. **Recent Tickets & Attention-Required Worklists**:

- **Attention Required Tickets**: Dedicated list of tickets in `WAITING_FOR_REQUESTER` status with ticket number, summary, priority badge, updated date, and direct action link.

- **Recent Tickets List**: List of max 5 requester's recent tickets with status badge, priority badge, updated date, and direct detail link.

- **Quick Action Button**: `+ Create Ticket` leading to ticket creation form.

Dashboard Interaction States

- **Loading State**: Displays skeleton cards or animated loading indicators without triggering page layout shifts.
- **Empty State**: Displays `0` for empty metrics and clear explanatory text for empty lists (e.g. `"No recent tickets"`, `"No tickets require attention"`).
- **Forbidden State**: Displays safe `403 Access Denied` UI feedback if unauthorized access is attempted.
- **Failure State**: Displays generic retryable error alert banner with a `[ Retry ]` button.
- **Not Found State**: Displays safe `404 Ticket not found.` alert banner with prominent direct fallback button `[ Return to My Tickets → ]` allowing the user to gracefully navigate back to the fallback list view if a targeted ticket has been removed.

Requester Metric Drill-Down Navigation

- Clicking **My Open Tickets Card** → Navigates to `/tickets/my-tickets?currentStatus=open`.
- Clicking **Waiting for My Response Card** → Navigates to `/tickets/my-tickets?currentStatus=WAITING_FOR_REQUESTER`.
- Clicking **In Progress Card** → Navigates to `/tickets/my-tickets?currentStatus=IN_PROGRESS`.
- Clicking **Recently Resolved Card** → Navigates to `/tickets/my-tickets?currentStatus=RESOLVEDresolvedWithinDays=30`.
- Clicking any ticket row in **Recent Tickets** or **Attention Required** → Navigates directly to `/tickets/rid`.

---

4. Ticket Detail Actions Taken Workspace

Located inside Ticket Detail (`/staff/tickets/rid` or `/tickets/rid`), positioned below Ticket Information and above Public Comments / Internal Notes.

Components & Controls

1. **Workspace Header**: Section title `Actions Taken Work Log` with action counter badge (`X actions recorded`).

2. **Create Action Form** (Collapsible / Modal):

- **Description**: Textarea, 1 to 2000 chars (validated after trimming whitespace), mandatory.

- **Result**: Textarea, 1 to 2000 chars (validated after trimming whitespace), mandatory.

- **Assignee Selector**: Dropdown listing active IT Staff and Administrator accounts (`isActive = true AND role IN ('IT_STAFF', 'ADMINISTRATOR')`). The Assignee Selector is optional and provides an explicit `Unassigned` state (`null`). Inactive/Requester users excluded.

- **Follow-Up Required Switch**: Checkbox (`Follow-up needed`).

EVD-QA-05: Rendered UI Specification establishing Zen Green color tokens, component typography, and responsive rules.

3. **Actions Taken Feed List**:

- Each action rendered in a structured card with `Action Date/Time`, `Performed By` badge (showing name + role), `Assignee` tag, status badge (`PENDING`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`), description, result, follow-up note (if present with amber warning border), and attachment notes.

- Actions ordered chronologically by `createdAt ASC`, tie-break `id ASC`.

- Action controls for authorized IT Staff and Administrators: Status transition controls enforcing permitted next statuses per Section 8.2 (`PENDING` → `[ Start Work ]` / `[ Complete Action ]` / `[ Cancel Action ]`; `IN_PROGRESS` → `[ Complete Action ]` / `[ Cancel Action ]`; terminal states `COMPLETED` and `CANCELLED` render no status transition buttons) and `[ Edit ]`. `COMPLETED` and `CANCELLED` `ActionTaken` records disable status transition buttons, but permit content editing via `[ Edit ]` modal using optimistic concurrency (`expectedUpdatedAt`), unless parent Ticket is `CANCELLED`.

- On `CANCELLED` tickets, all Action Taken creation, editing, and status controls are hidden and disabled.

Action Submission Reliability

- The client MUST generate one unique `Idempotency-Key` for each intended Action submission.
- The same key MUST be reused when the same submission is retried after a recoverable network failure.
- A new key MUST be generated only for a new user-intended submission.
- The Save Action control SHOULD be disabled/debounced while the initial submission is in flight.
- Backend idempotency remains the authoritative duplicate-submission protection.

Requester Read-Only View

- Requesters can view Actions Taken for Tickets they are authorized to access (owned tickets).
- Requesters see Actions Taken in read-only mode.
- Create Action Form, Edit Action, Assignee Selector, Complete Action, Cancel Action, and Status Transition controls are strictly NOT rendered for Requesters.

5. Ticket Workflow & Resolution UI

Status Controls

- Render only permitted next-status controls according to Section 8.1 Ticket Status Transition Matrix.
- Disabled or hidden UI controls MUST NOT be treated as the security boundary; server-side validation strictly enforces authorization and transitions.
- `RESOLVED` status transition button requires satisfying the server-side Resolution Gate (>= 1 COMPLETED ActionTaken or non-empty staff `resolutionNote`).
- Resolution Gate failures display a clear inline error message (`"Cannot resolve ticket without at least one completed Action Taken or formal resolution note."`).
- `CANCELLED` renders as a terminal state and exposes no further status controls.
- `409 Conflict` (`STALE_UPDATE`) displays a stale-update alert banner with a `[ Refresh / Reload ]` button to fetch latest ticket state.

Resolution Note Input Contract

- When an IT Staff member or Administrator selects the `RESOLVED` status transition, the UI renders a **Resolution Note** text area (1 to 2000 characters).
- If the Ticket already has at least 1 completed `ActionTaken` record, the `resolutionNote` input is optional.
- If the Ticket has 0 completed `ActionTaken` records, the `resolutionNote` input is MANDATORY (validated with trimmed whitespace).
- If submitted without a completed `ActionTaken` and with an empty `resolutionNote`, the UI displays an inline validation error (`"Cannot resolve ticket without at least one completed Action Taken or formal resolution note."`) and blocks request submission.

Requester Indicators

- Requester "Problem Appears Resolved" indication (`POST /api/tickets/:id/resolve-indicator`) is advisory and MUST NOT change the Ticket status to `RESOLVED`.
- The indicator records `requesterResolvedIndicatedAt` and allows IT Staff or Administrators to review the Requester's indication before executing a formal Ticket status transition.

Role Behavior

- IT Staff and Administrators follow the operation-level authorization matrix and can execute formal status transitions.
- Requesters CANNOT execute formal ticket status transitions; they may post public comments or send the advisory "Problem Appears Resolved" indication.

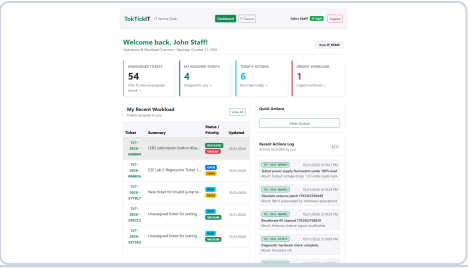
6. Responsive Breakpoints & Viewport Rules

The interface MUST render seamlessly across three standard viewports:

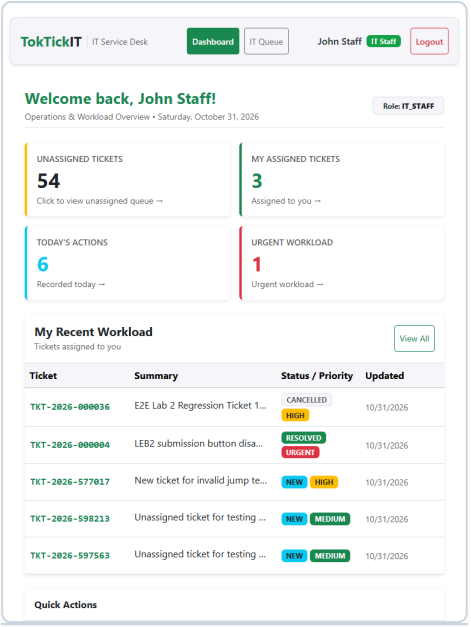
Viewport Name	Target Dimensions	Responsive Adaptations
Desktop	`1920×1080` (16:9)	Full 4-card metric grid, side-by-side dashboard columns, full data tables with sort headers.
Tablet	`768×1024` (3:4)	2×2 metric grid, stacked dashboard columns, horizontally scrollable data tables.
Mobile	`375×812` (9:19.5)	Single-column metric card stack, touch-friendly action buttons, responsive ticket & action cards.

7. Visual & Accessibility QA Checklist

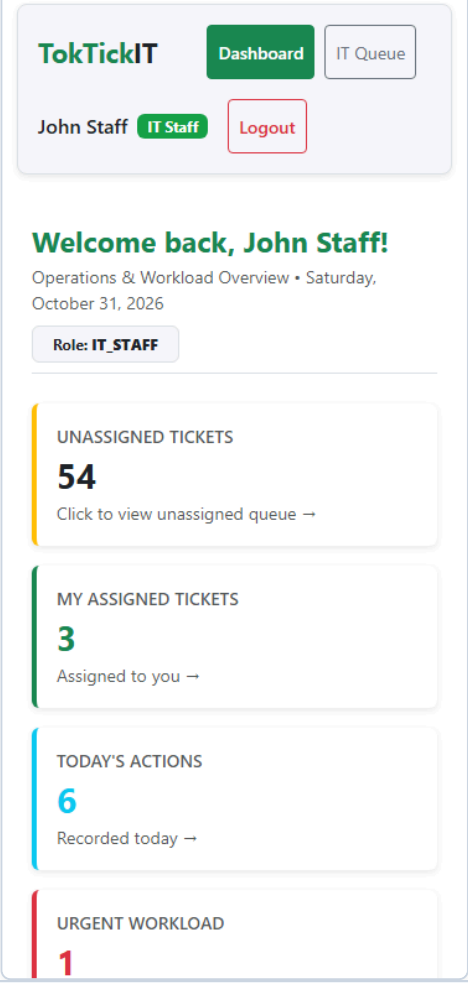
- [x] **No Clipping**: No clipping was observed in the required Lab 4 screens during visual inspection. Automated verification checks page-level horizontal overflow, with targeted bounding-box checks for representative dashboard cards and Actions Taken form controls.
- [x] **No Overlap**: No unintended overlap was observed in the required Lab 4 screens during visual inspection. Automated bounding-box checks cover representative navigation controls, all four Staff Dashboard metric cards, and Actions Taken form fields.
- [x] **No Unintended Horizontal Overflow**: Page body width remains strictly bounded to viewport width (`document.documentElement.scrollWidth <= window.innerWidth + 1`) without relying on body-level overflow clipping (`overflow-x: hidden`). Data tables use isolated responsive scroll wrappers (`overflow-x: auto`).
- [x] **Visible Focus States**: Representative required interactive controls have a visible Zen Green keyboard focus ring (`outline: 2px solid #198754 !important; outline-offset: 2px;`) verified during keyboard traversal.
- [x] **Non-Color Status & Priority Cues**: Statuses, priorities, and roles use distinct readable textual labels alongside color-coded badges, ensuring full accessibility for color-vision-deficient users.
- [x] **Mobile Touch Targets**: Required mobile interactive controls verified in the main navigation and Actions Taken form enforce minimum touch dimensions of `>= 44×44px` on mobile viewports (`<= 768px`).
- [x] **Semantic Labels & ARIA Attributes**: Representative required forms (Login, Create Ticket, Actions Taken) use explicit label-to-control associations (`for`), landmark regions, and valid ARIA roles (`role="region"`, `role="button"`, `aria-current="page"`).
- [x] **Editable vs Read-Only Field Clarity**: Read-only fields (e.g. Requester Actions Taken feed, Ticket ID, timestamps) are visually distinguished from editable controls (Create/Edit Action modals, Resolution Note textarea).
- [x] **Clear Validation & Alert Placement**: Required representative validation states (Login error banner, Actions Taken inline feedback,



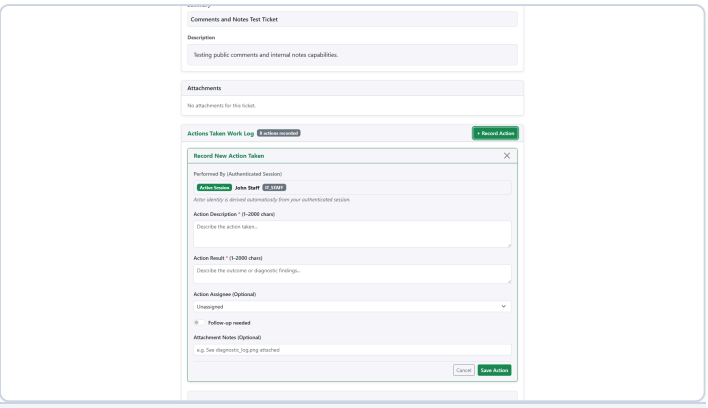
EVD-QA-01: Desktop Viewport (1920×1080)



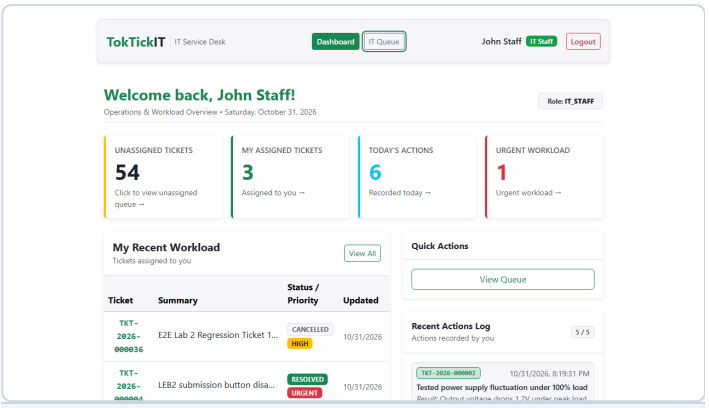
EVD-QA-02: Tablet Viewport (768×1024)



EVD-QA-03: Mobile Viewport (375×812)



EVD-QA-07: Dedicated Actions Taken Multi-Device Responsive Audit & Touch Target Inspection.



EVD-QA-08: Accessibility Focus Rings (`2px solid #198754`) with 2px offset and full keyboard tab navigation.

Accessibility & Quality Check	Specification Invariant	Automated / Visual Verification	Status
Touch Target Size	Minimum 44×44px on mobile viewports	`TEST-QA-05` (Mobile inputs & buttons >= 44×44px)	PASS
Focus Indicators	High-contrast 2px solid #198754 outline	`TEST-QA-06` (Keyboard tab sequence without mouse)	PASS
Color Accessibility	Non-color textual status/priority cues	`TEST-A11Y-01` (All badges have explicit text)	PASS
Layout & Overflow	Zero page-level horizontal overflow	`TEST-RESP-01` (0 horizontal scroll at 375px)	PASS
Field Collisions	No clipping or overlap on narrow screens	`TEST-QA-05` (Bounding-box non-overlap checks)	PASS
Form Preservations	Safe errors preserve user form inputs	`TEST-FAIL-01` (Inputs intact on 400 error)	PASS